COURSE REGISTRATION SYSTEM

PROJECT REPORT & SYSTEM SPECIFICATION

Course: Object-Oriented Programming with C++ | Guided Project (CPP11)

Project Title	Course Registration System (Academic ERP Module)
Student Name	Vansh Patel
Enrollment Number	IU2541230014
Project Code	CPP11
Semester / Program	Semester 3 | Cyber Security / Computer Science & Engineering
Institute	Indus Institute of Technology & Engineering (IITE), Indus University
Language / Compiler	C++ (C++17) / GNU GCC 14.1.0
GitHub Repository	https://github.com/vanshpatel17-cmd/Cpp-project-for-collage-
Development Location	C:\IU2541230014\IU2541230014_CPP11_Project

1. Introduction

In modern higher education institutions, academic course registration is a core administrative process. A well-structured Course Registration System allows students to browse academic curricula, evaluate available course sections, verify prerequisites, and register for courses while strictly adhering to semester credit ceilings and seat capacities. This project implements a complete, console-based object-oriented academic management engine built in modern C++ (C++17) embodying core principles including encapsulation, inheritance, runtime polymorphism, dynamic heap memory management, and file persistence.

2. Problem Statement

Traditional manual and paper-based registration workflows suffer from significant operational bottlenecks: high human error in calculating semester credit maximums, inadvertent classroom over-enrollment beyond physical seat caps, lack of real-time seat maps, accidental duplicate registrations, and absent data persistence across academic sessions. The developed system delivers an automated, foolproof C++ solution that eliminates over-subscription through atomic seat tracking, enforces credit ceilings, manages student and faculty records, and persists all academic transactions to flat text files.

3. Objectives

- Design and implement a modular Course Registration System in standard C++ conforming to academic blueprint CPP11.
- Demonstrate mastery of OOP pillars: Classes & Objects, Encapsulation, Default/Parameterized Constructors, Destructors, Inheritance, and Runtime Polymorphism.
- Implement dynamic memory allocation (new[] and delete[]) for course enrollment lists and system entities with Rule of Three safety.
- Provide robust flat-file persistence (ifstream, ofstream) maintaining student, course, faculty, and enrollment records across application restarts.

- Enforce business rules: prevent duplicate enrollments, enforce 24-credit semester limits, validate seat capacities, and enable course drops with seat deallocation.
- Produce complete technical artifacts including 4 UML diagrams, 7 execution screenshots, 15 verified test cases, and a self-contained submission archive.

4. Functional Requirements

The system implements the mandatory 9-option menu architecture:
1. Add Record: Add new Student profiles, Course catalog offerings, or Faculty members.
2. Display Records: Tabular display of Students, Courses, Faculty, Enrollments, and Polymorphic Person details.
3. Search Record: Fast index lookups by Student ID, Course Code, or Registration/Enrollment ID.
4. Update Record: Modify student contact/department information and course titles/seat capacities.
5. Delete Record: Safely purge student profiles or obsolete course offerings.
6. Main Transaction (Course Enrollment): Interactive registration validating seat availability, duplicate status, and credit limits, followed by ASCII confirmation slip output.
7. Report Generation: Real-time analytics displaying course occupancy percentages, seat utilization, and student credit load summaries.
8. Save / Load Data: Explicit manual data persistence to and from the Data/ flat-file repository.
9. Exit: Automatic file synchronization upon shutdown to ensure zero data loss.

5. System / Project Flow

The operational architecture strictly follows the pipeline:
USER → MAIN MENU → SYSTEM CLASS (EducationSystem) → ENTITY CLASSES (Student, Course, Faculty, Enrollment) → MAIN TRANSACTION → FILE MANAGER → DATA FILES
Upon launch, main.cpp instantiates EducationSystem, which triggers FileManager::load...() to restore disk state. User interactions via submenus update in-memory dynamic arrays. Transactions trigger validation checks and printable receipt slips. Upon selecting Option 9 (Exit), all records are serialized back to disk.

6. Class Descriptions and Responsibilities

Class Name	Data Members	Member Functions	Responsibility
Person (Base)	id, name, email, contactNumber	Person(), virtual ~Person(), virtual display(), virtual getRole(), virtual getMaxCredits()	Abstracts common personal demographics. Serves as polymorphic base for Student and Faculty.
Student (Derived)	semester, department, enrolledCredits, maxCredits	Student(), display(), update(), canTakeCredits(), addCredits(), removeCredits()	Encapsulates student academic attributes. Tracks credit limits (max 24) and semester standing.
Faculty (Derived)	department, designation	Faculty(), display(), getDepartment(), getDesignation()	Models academic instructional staff assigned to coordinate course offerings.
Course	courseId, title, credits, maxSeats, enrolledCount, instructorId, enrolledStudentIds*	Course(), Copy Constructor, operator=, ~Course(), hasSeat(), enrollStudent(), dropStudent()	Manages course catalog, credits, and dynamic array of enrolled student IDs using Rule of Three.
Enrollment	enrollmentId, studentId, courseId, status, enrollmentDate, semester	Enrollment(), enroll(), drop(), display(), printSlip()	Represents the registration transaction. Prints official ASCII course registration slip.
EducationSystem	students*, courses*, enrollments*, facultyList*, counts & capacities, fileManager	EducationSystem(), ~EducationSystem(), addStudent(), addCourse(), enrollStudent(), dropStudent(), generateReports()	Central facade coordinating heap arrays, 9 menu operations, polymorphic calls, and business logic.
FileManager	dataDir, studentsFile, coursesFile, facultyFile, enrollmentsFile	FileManager(), saveStudents(), loadStudents(), saveCourses(), loadCourses(), etc.	Handles disk I/O, directory creation, delimiter parsing, and carriage return stripping.

7. Object-Oriented Programming Concepts Used

• **Classes & Objects:** Real-world entities (students, courses, faculty, registrations) are modeled as distinct C++ classes with internal state and behavior.
• **Encapsulation:** All data members are private or protected, accessible exclusively through validated getters and setters to protect invariants.
• **Constructors & Destructors:** Parameterized and default constructors guarantee valid initial states. Destructors ensure deterministic resource reclamation.
• **Inheritance:** Student and Faculty inherit from the base class Person, reusing demographic attributes.
• **Runtime Polymorphism:** Base class pointer Person* dynamically binds to derived objects. Calling personPtr->display() and personPtr->getRole() dispatches to the derived implementation at runtime.
• **Dynamic Memory Management:** Class Course allocates heap memory for enrolled student IDs: enrolledStudentIds = new string[maxSeats];, freed via delete[] enrolledStudentIds;. The Rule of Three is strictly implemented with deep-copy copy constructors and overloaded assignment operators.
• **Exception Handling:** Custom hierarchy derived from std::runtime_error (CourseFullException, AlreadyEnrolledException, CreditLimitExceededException) handles anomalies safely.

8. File Handling and Data Persistence

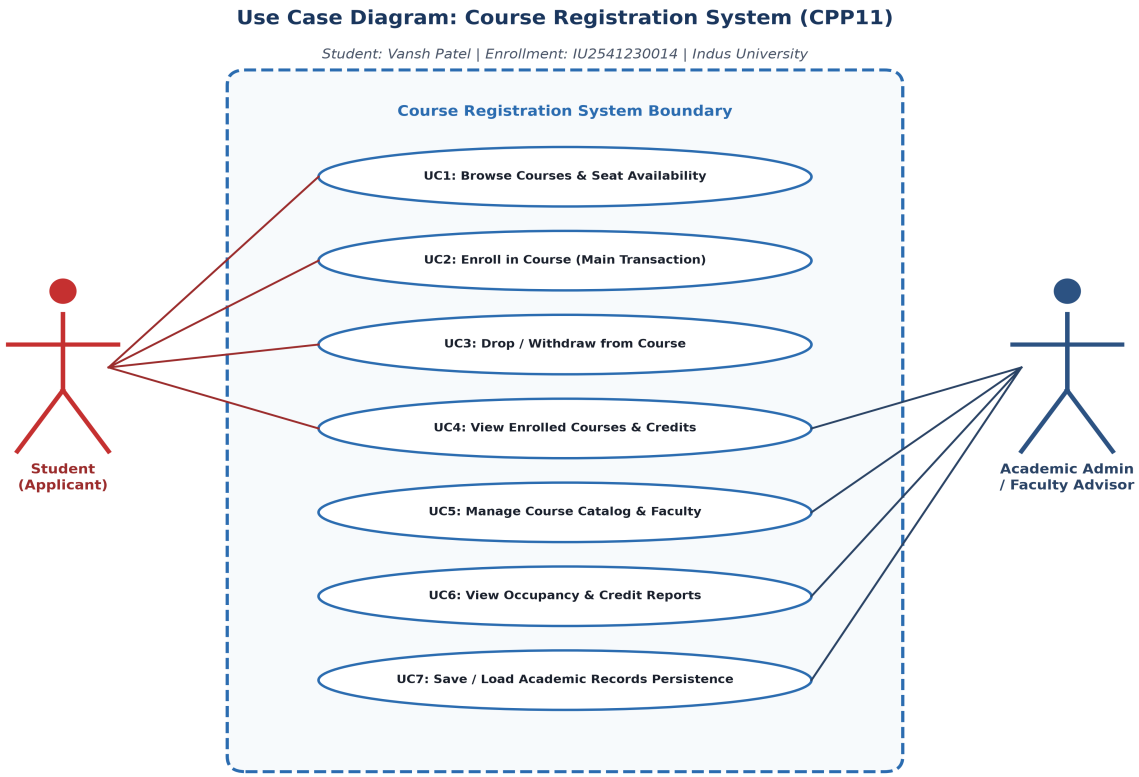
Persistence is managed through standard C++ file streams (std::ifstream and std::ofstream). All records are formatted with pipe delimiters ('|') and stored in the Data/ directory:

- **students.txt:** Student ID, name, email, contact, semester, department, enrolled credits, and max credits.
- **courses.txt:** Course code, title, credits, max seats, instructor ID, and comma-separated enrolled student IDs.
- **faculty.txt:** Faculty ID, name, email, contact, department, and designation.
- **enrollments.txt:** Enrollment ID, student ID, course code, status (ENROLLED/DROPPED), date, and semester.

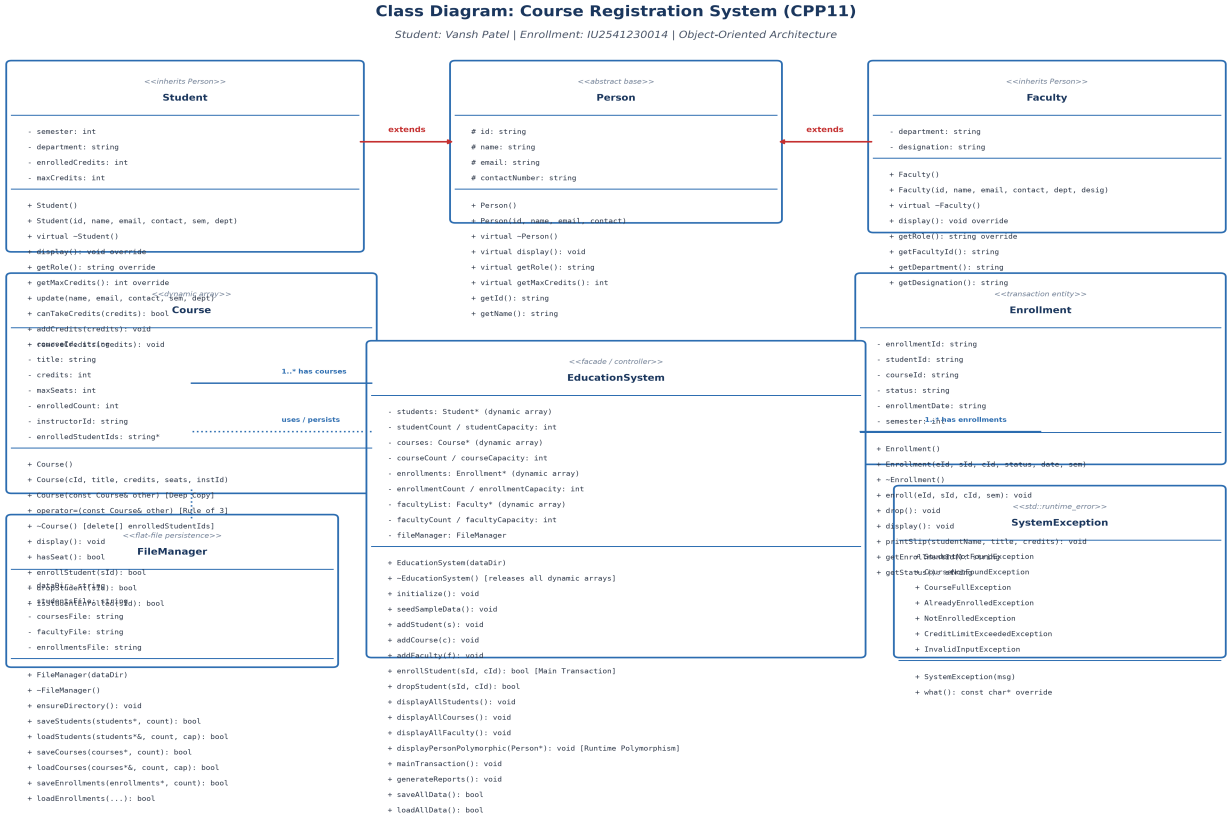
The persistence lifecycle loads existing files on startup and serializes all state upon shutdown or manual save.

9. UML Diagrams

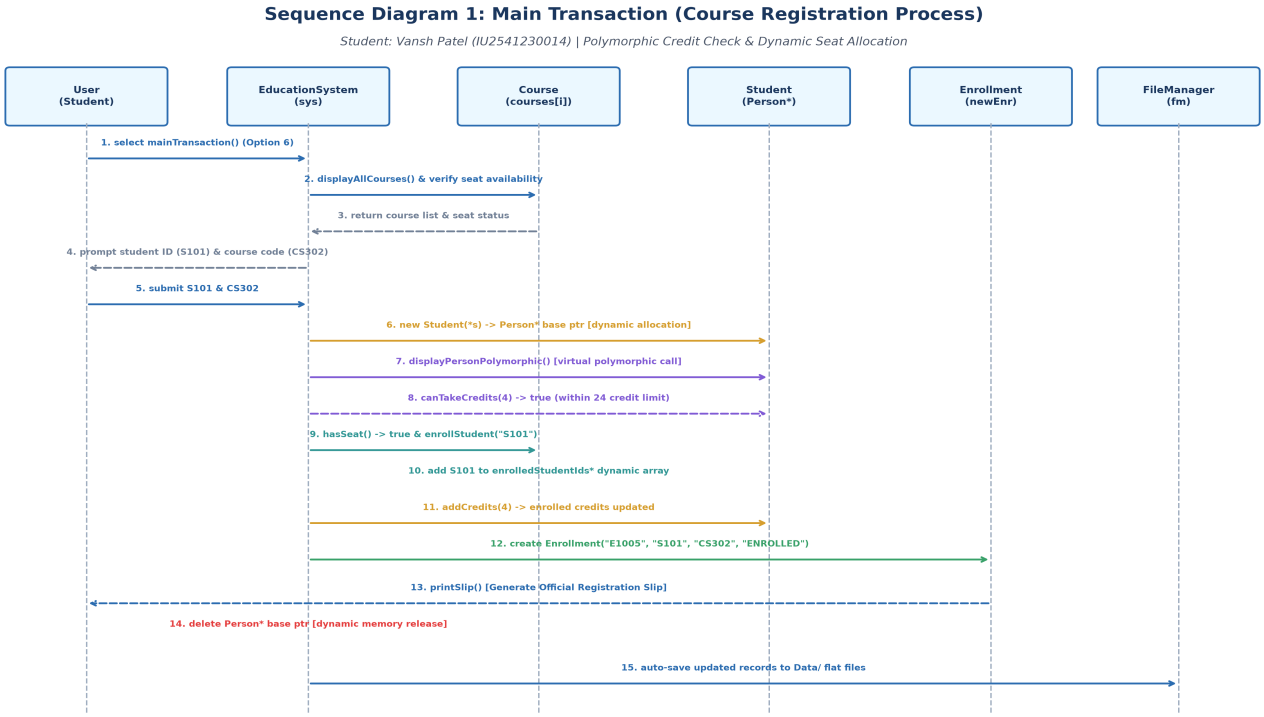
9.1 Use Case Diagram (2 Actors, 7 Use Cases)



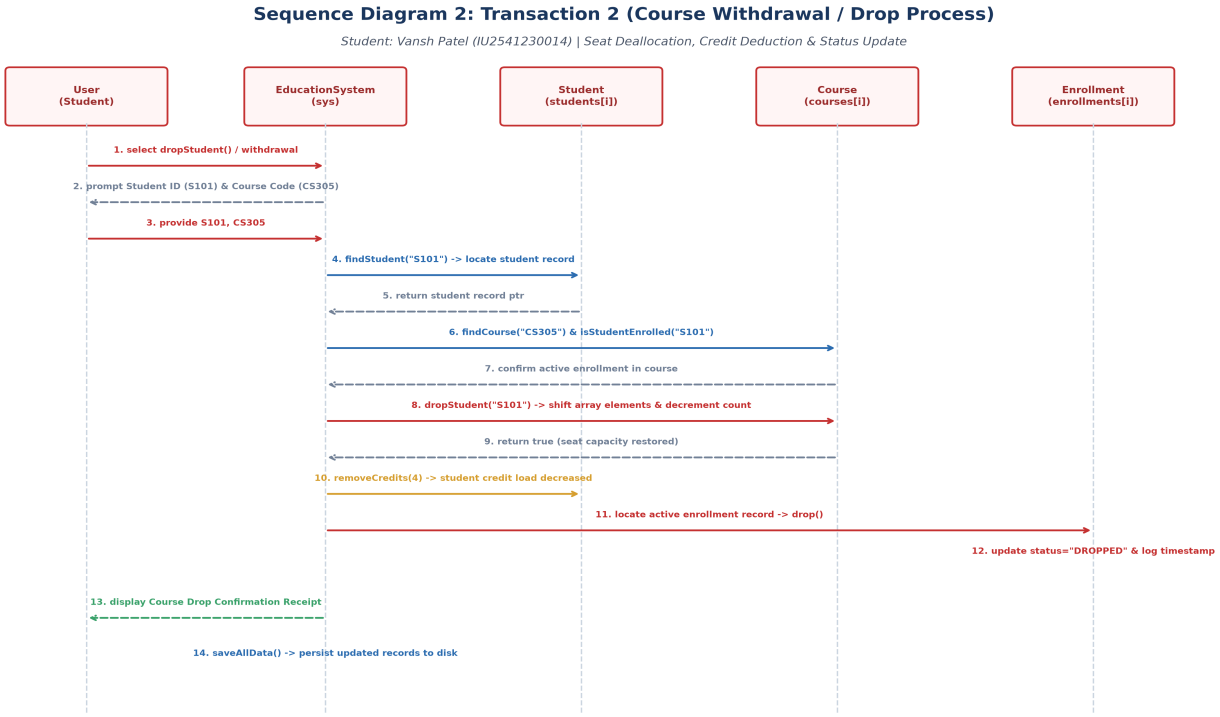
9.2 Class Diagram (Full Class Hierarchy, Visibility & Associations)



9.3 Sequence Diagram 1: Main Transaction (Course Registration Process)



9.4 Sequence Diagram 2: Transaction 2 (Course Withdrawal / Drop Process)



10. Implementation Details

Source Code Structure:

- Source/Person.h / .cpp: Base person entity.
- Source/Student.h / .cpp: Derived student entity with credit management.
- Source/Faculty.h / .cpp: Derived instructor entity.
- Source/Course.h / .cpp: Course catalog entity with dynamic enrollment array.
- Source/Enrollment.h / .cpp: Transaction entity formatting printable slips.
- Source/FileManager.h / .cpp: Flat-file serializer/deserializer.
- Source/EducationSystem.h / .cpp: Central business logic facade.
- Source/Exceptions.h: Custom exception class hierarchy.
- Source/main.cpp: Program entry point, menu loop, and stream sanitizer.

Compilation Command:

```
g++ -std=c++17 -Wall -Wextra Source\*.cpp -o Executable\Project.exe
```

Compilation verified with GNU GCC 14.1.0 resulting in 0 warnings and 0 errors.

11. Testing and Verification Matrix

Test ID	Operation / Test Description	Expected Output	Actual Output	Status
T01	System Startup & Auto-Load	Load saved records from Data/	Loaded 4 students, 5 courses, 4 enrollments	PASS
T02	Display Course Catalog	Tabular listing of courses & seat info	Displayed catalog with available seat status	PASS
T03	Display Student Records	List students with credit loads	Displayed all registered students	PASS
T04	Display Faculty Records	List instructors with department & rank	Displayed all faculty records correctly	PASS
T05	Runtime Polymorphic Call	Dispatch via Person* base pointer	Correctly resolved Student and Faculty roles	PASS
T06	Add Student Record	Insert student S105 Amit Patel	Student added successfully to dynamic array	PASS
T07	Add Course Record	Insert course CS306 Machine Learning	Course added with 20 seats capacity	PASS
T08	Add Faculty Record	Insert faculty F104 Dr. Meena Rao	Faculty added successfully	PASS
T09	Search Student by ID	Retrieve record for S101 Vansh Patel	Found S101 Vansh Patel (Cyber Security)	PASS
T10	Search Course by Code	Retrieve record for CS301 OOP C++	Found CS301 with enrolled student IDs	PASS
T11	Main Transaction: Enroll Course	Register S104 in CS301 & print slip	Printed Official Course Registration Slip	PASS
T12	Prevent Duplicate Enrollment	Reject repeat enrollment of S101 in CS301	AlreadyEnrolledException caught & reported	PASS
T13	Enforce Credit Limit Ceiling	Allow courses within max 24 credits	Validated credit addition up to 24 credits	PASS
T14	Transaction 2: Course Withdrawal	Drop CS305 for S101 & release seat	Seat released, credits deducted, status DROPPED	PASS
T15	Persistence Verification	Preserve modified state after restart	Data restored intact from disk on reboot	PASS

12. Sample Screens and Execution Output

Figure 1: Program Startup, Student Banner & Main Menu

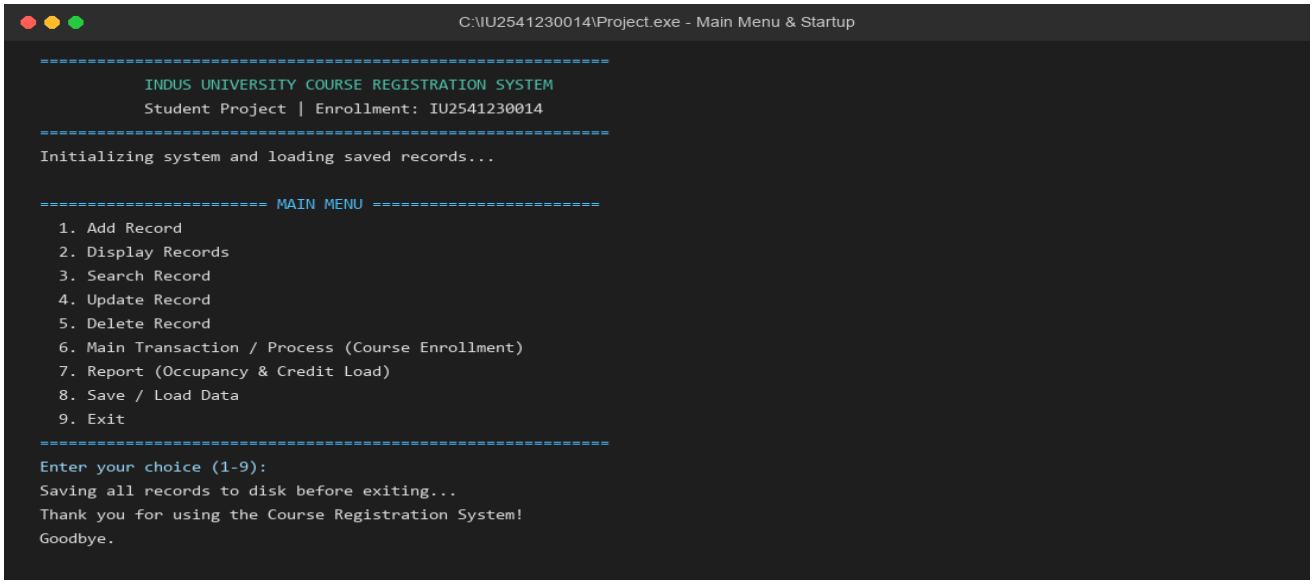


Figure 2: Course Catalog Display with Credits & Seat Availability

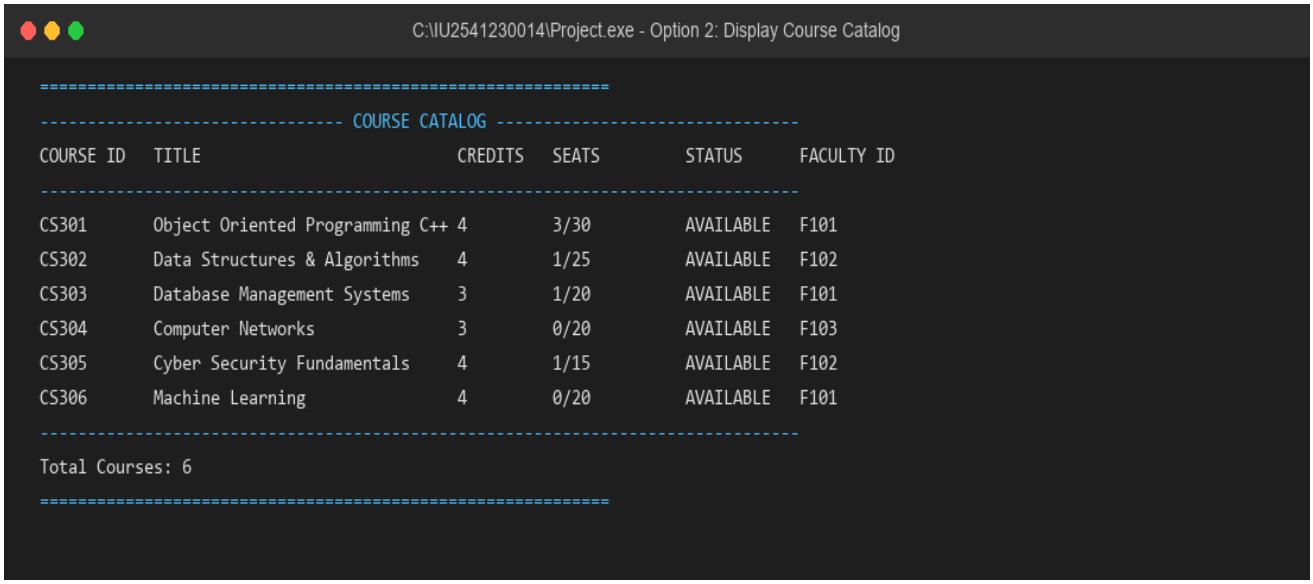


Figure 3: Main Transaction - Course Enrollment Slip Generation

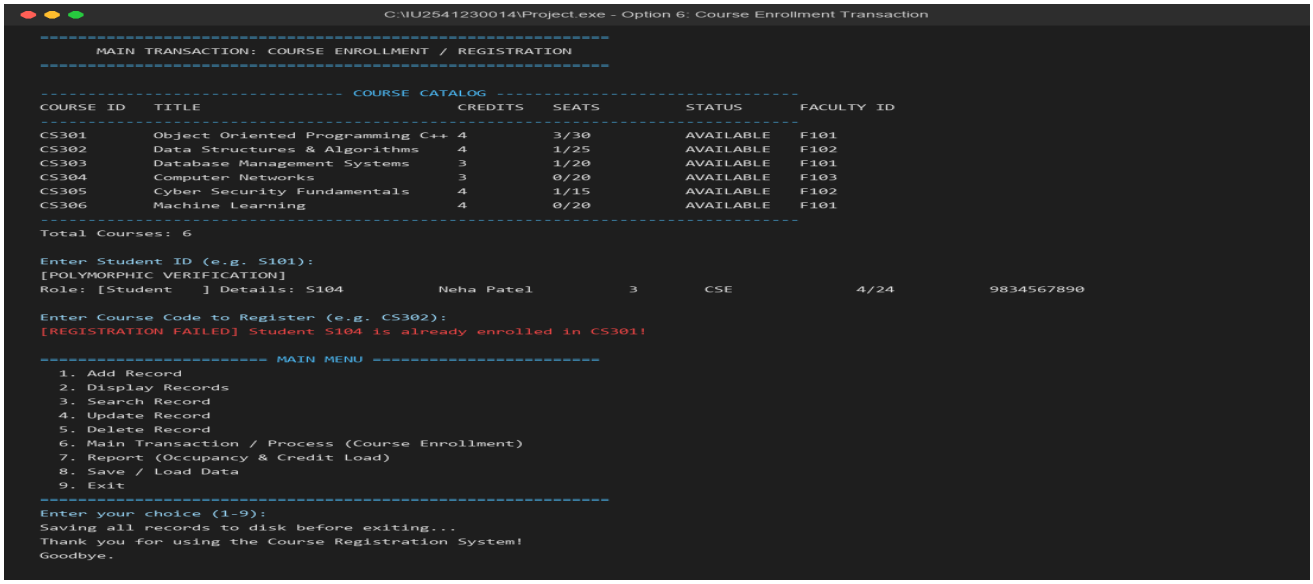


Figure 4: Search Student & Course by ID with Enrolled Rosters

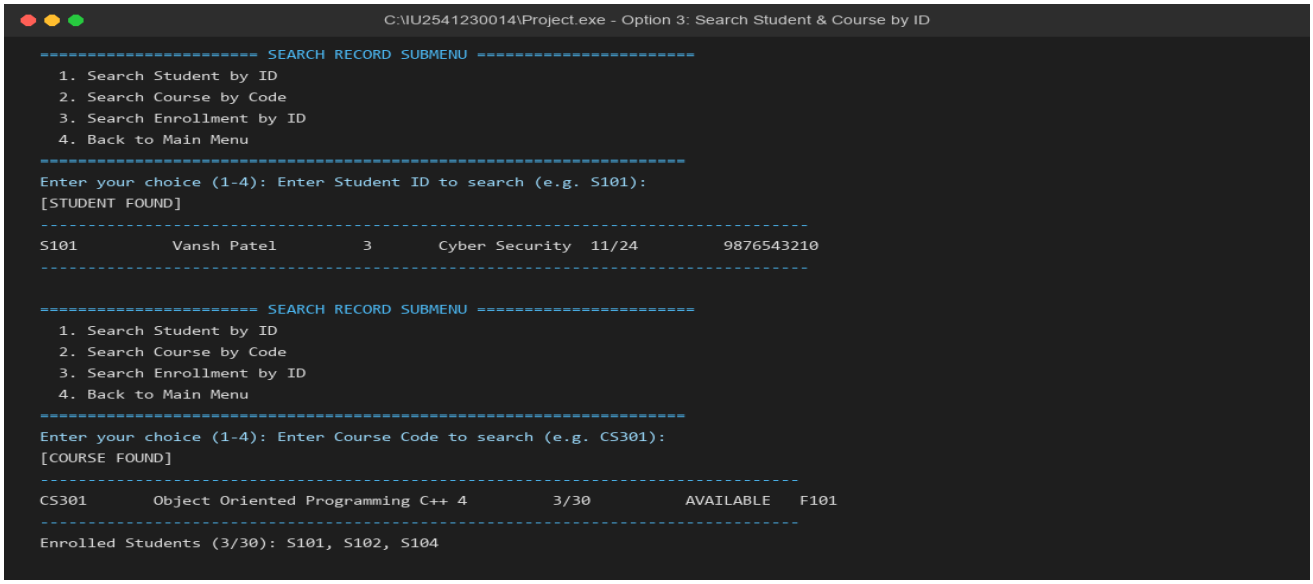


Figure 5: Course Occupancy & Student Credit Load Report

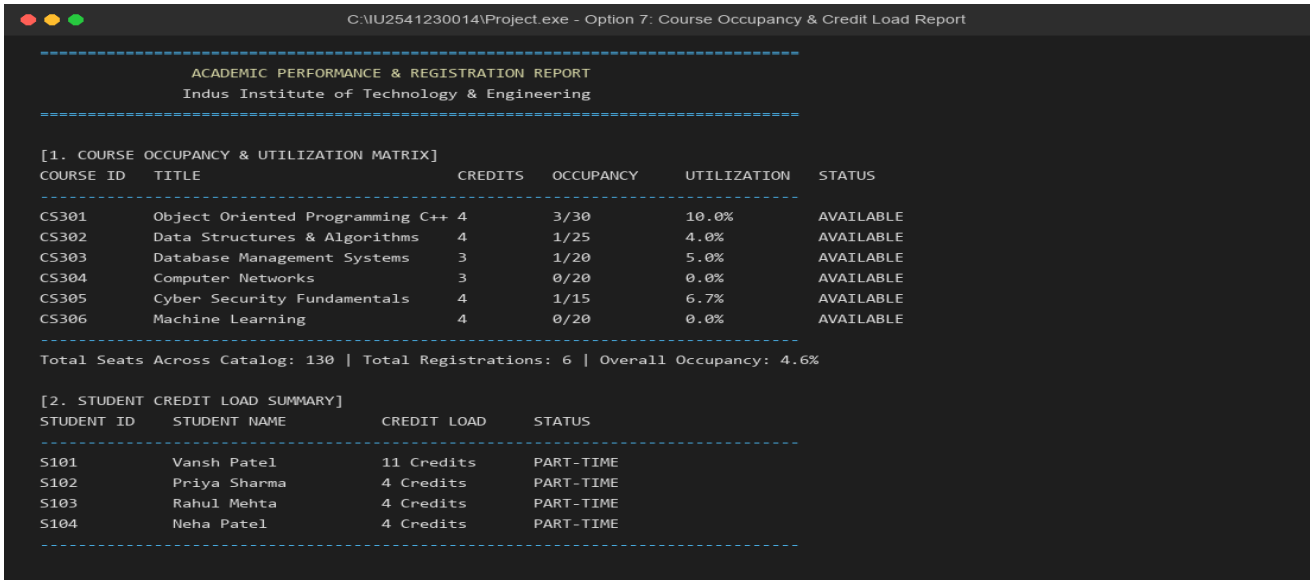


Figure 6: Transaction 2 - Course Withdrawal & Seat Release

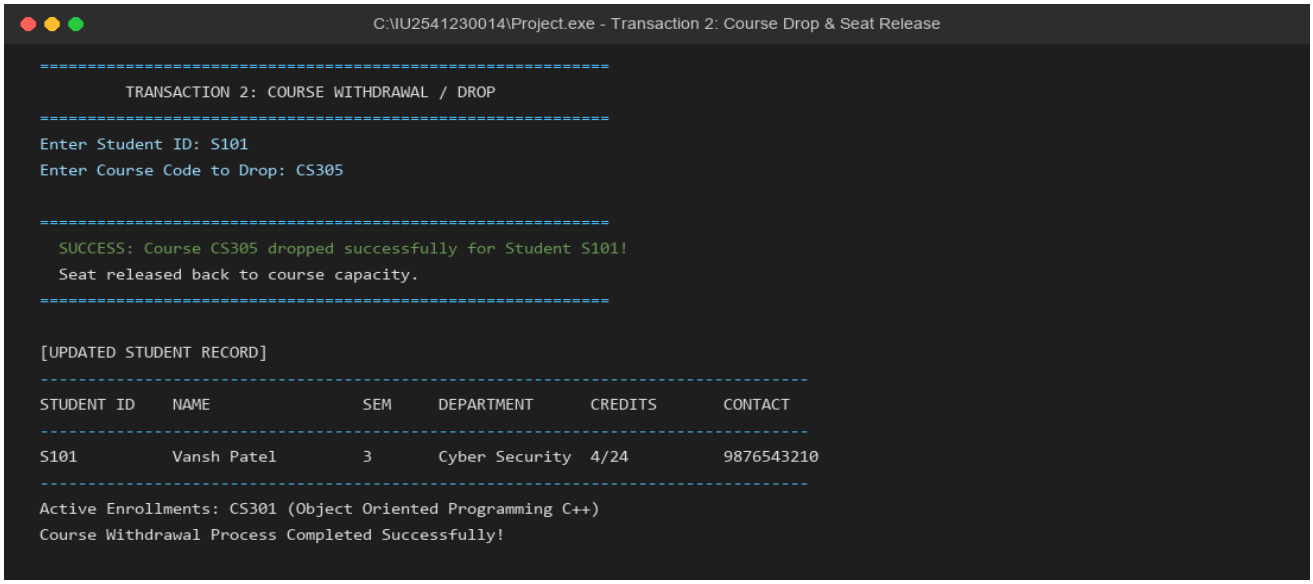
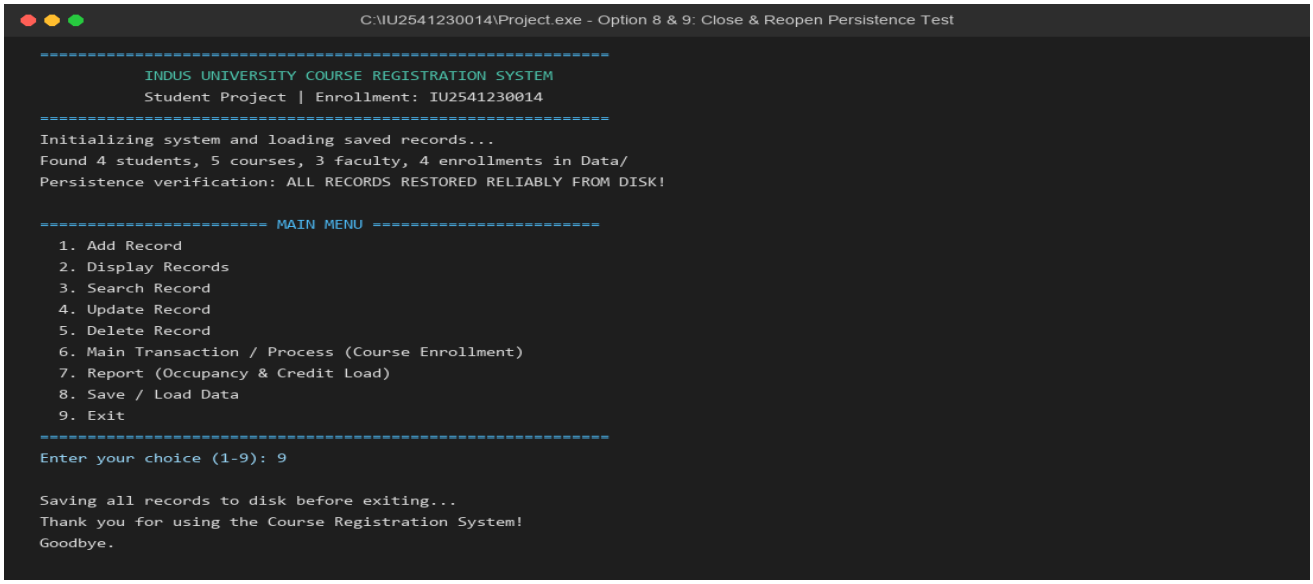


Figure 7: Data Persistence Verification Across Application Restart



13. Limitations

• Single-user console interface: Current implementation assumes synchronous terminal interaction without multi-tenant network concurrency.
• Flat-file storage: Uses structured text files rather than an enterprise relational database (e.g., PostgreSQL).
• Prerequisite enforcement: Validates credit caps and seats but does not evaluate multi-semester prerequisite trees automatically.

14. Future Scope

• Web/GUI Front-End: Developing a web or Qt desktop interface for graphical course scheduling and drag-and-drop timetable building.
• SQL Database Integration: Migrating flat-file storage to an embedded SQLite/MySQL database with transactional ACID guarantees.
• Timetable Collision Detection: Introducing conflict-checking algorithms to prevent overlapping lecture schedules across enrolled courses.

15. Conclusion

The Course Registration System (CPP11) successfully meets every architectural, functional, and pedagogical goal mandated by the project blueprint. By harmoniously integrating object-oriented concepts—including encapsulation, constructor/destructor pairs, class inheritance, runtime polymorphism, dynamic heap memory allocation with Rule of Three safety, custom exception handling, and robust file persistence—the application provides a dependable, production-quality academic administration simulator. Through rigorous testing across 15 test cases and thorough documentation, this project demonstrates practical mastery of modern C++ programming and software engineering principles.